

TP INITIATION A LA CARTE ESP32

Durée : 3 heures • Niveau : Initiation

Classe : _____

Professeur : Korota Arsène COULIBALY



Année Universitaire 2025-2026

20 août 2026

Objectifs et matériel

- Lire température et humidité depuis un DHT11 avec l'ESP32 dans l'IDE Arduino, publier sur MQTT, et s'abonner pour piloter une LED via Node-RED Dashboard.
- Installer les bibliothèques DHT d'Adafruit et Adafruit Unified Sensor, configurer Mosquitto et Node-RED + Dashboard, puis vérifier la chaîne de bout en bout.

Architecture cible

- L'ESP32 publie température et humidité sur `esp32/temperature` et `esp32/humidity` et s'abonne à `esp32/output` pour recevoir on/off afin de piloter une LED.
- Node-RED reçoit et affiche les mesures (gauge/chart) et envoie on/off via un switch UI vers `esp32/output` pour commander la LED de l'ESP32.

Câblage DHT11 et LED

- DHT11 : VCC → 3.3 V, GND → GND, DATA → GPIO 27 (ou autre GPIO numérique libre). Ajouter une résistance de tirage vers VCC si capteur nu (4.7–10 kΩ) ; les modules DHT11 à 3 broches l'intègrent souvent.
- LED : conserver la LED sur GPIO 4 avec résistance 220 Ω en série vers l'anode et retour cathode vers GND pour un test de commande MQTT simple.

Installer la prise en charge ESP32

1. Ouvrir l'IDE Arduino, aller dans **Fichier > Préférences**, puis dans *Additional Boards Manager URLs*, coller l'URL officielle du package ESP32 d'Espressif indiquée par la documentation Arduino/ESP32, puis valider avec **OK**.
2. Ouvrir le Gestionnaire de cartes via **Outils > Type de carte > Gestionnaire de cartes...**, chercher "ESP32", puis installer "*esp32 by Espressif Systems*".
3. Redémarrer l'IDE Arduino, puis vérifier que les cartes ESP32 apparaissent dans **Outils > Type de carte** (par exemple *DOIT ESP32 DEVKIT V1*).

Méthode IDE Arduino 2.x

1. Dans Arduino IDE 2, aller dans **Fichier > Préférences** et coller l'URL officielle ESP32 dans *Additional Boards Manager URLs* si elle n'est pas déjà présente.
2. Ouvrir le Gestionnaire de cartes depuis l'icône dédiée ou via **Outils > Type de carte > Gestionnaire de cartes...**, rechercher "ESP32" et installer la version "*esp32 by Espressif Systems 3.x*".
3. Redémarrer l'IDE, puis sélectionner la carte ESP32 et le port série via **Outils**, ou par le sélecteur rapide *Select other board and port*.

Astuces et cas fréquents

- Si une autre URL (ex. ESP8266) est déjà présente, séparer les entrées par une virgule dans *Additional Boards Manager URLs*.
- En cas d'échec de téléchargement du package, réessayer après redémarrage de l'IDE et du réseau, ou utiliser l'URL miroir officielle alternative, puis relancer l'installation via le Gestionnaire de cartes.
- Sous Windows, si le port n'apparaît pas, installer le pilote USB–UART adapté (ex. CP210x/CH340) et reconnecter l'ESP32 avant de re-sélectionner le port dans **Outils > Port**.

Vérifier l'installation

- Brancher l'ESP32 en USB, puis choisir la carte (ex. *DOIT ESP32 DEVKIT V1*) et le **Port** dans **Outils**.
- Ouvrir un exemple de test (par exemple *WiFiScan* dans **Fichier > Exemples > WiFi (ESP32)**), téléverser et vérifier la sortie dans le Moniteur série à 115200 bauds.

Code ESP32 (DHT11 + MQTT + LED)

Renseigner le SSID/mot de passe Wi-Fi et l'IP du broker MQTT, sélectionner DHT11 et la broche GPIO du DATA (ici 27), puis téléverser le code suivant sur l'ESP32.

```
1 // ESP32 + DHT11 + MQTT + Node-RED
2 #include <WiFi.h>
3 #include <PubSubClient.h>
4 #include <Adafruit_Sensor.h>
5 #include <DHT.h>
6
7 // Wi-Fi
8 const char* ssid      = "VOTRE_SSID";
9 const char* password  = "VOTRE_MDP_WIFI";
10
11 // MQTT
12 const char* mqtt_server = "IP_DU_BROKER";      // ex: "192.168.1.100"
13 const int   mqtt_port   = 1883;
14 const char* TOPIC_CMD   = "esp32/output";
15 const char* TOPIC_T     = "esp32/temperature";
16 const char* TOPIC_H     = "esp32/humidity";
17
18 // LED sur GPIO 4
19 const int LED_PIN = 4;
20
21 // DHT11
22 #define DHTPIN  27
23 #define DHTTYPE DHT11
24 DHT dht(DHTPIN, DHTTYPE);
25
26 WiFiClient espClient;
27 PubSubClient client(espClient);
```

```

28
29 unsigned long lastMsg = 0;
30 // DHT11   chantillonne   ~1 Hz; publier toutes ~2 s pour des
    valeurs fra ches
31 const unsigned long PUBLISH_MS = 2000;
32
33 void setup_wifi() {
34     WiFi.mode(WIFI_STA);
35     WiFi.begin(ssid, password);
36     while (WiFi.status() != WL_CONNECTED) { delay(300); }
37 }
38
39 void callback(char* topic, byte* payload, unsigned int length) {
40     String msg;
41     for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
42     if (String(topic) == TOPIC_CMD) {
43         if (msg == "on")    digitalWrite(LED_PIN, HIGH);
44         if (msg == "off")  digitalWrite(LED_PIN, LOW);
45     }
46 }
47
48 void reconnect() {
49     while (!client.connected()) {
50         // ClientID unique conseil
51         if (client.connect("ESP32Client")) {
52             client.subscribe(TOPIC_CMD);
53         } else {
54             delay(2000);
55         }
56     }
57 }
58
59 void setup() {
60     pinMode(LED_PIN, OUTPUT);
61     digitalWrite(LED_PIN, LOW);
62     Serial.begin(115200);
63     dht.begin();
64     setup_wifi();
65     client.setServer(mqtt_server, mqtt_port);
66     client.setCallback(callback);
67 }
68
69 void loop() {
70     if (!client.connected()) reconnect();
71     client.loop();
72
73     unsigned long now = millis();
74     if (now - lastMsg >= PUBLISH_MS) {
75         lastMsg = now;
76
77         float h = dht.readHumidity();

```

```

78     float t = dht.readTemperature(); // Celsius
79
80     if (isnan(h) || isnan(t)) {
81         // Lecture invalide; ignorer cette it ration
82         return;
83     }
84
85     char tbuf[8]; dtostrf(t, 1, 2, tbuf);
86     char hbuf[8]; dtostrf(h, 1, 2, hbuf);
87
88     client.publish(TOPIC_T, tbuf);
89     client.publish(TOPIC_H, hbuf);
90 }
91 }

```

Pourquoi 2 s d'intervalle Le DHT11 met à jour ses mesures environ à 1 Hz; attendre environ 2 s évite des valeurs répétées et respecte ses limitations de rafraîchissement.

Alternative de capteur La bibliothèque Adafruit DHT gère DHT11 et DHT22 avec la même API; seul DHTTYPE diffère, ce qui facilite l'adaptation si vous changez de capteur.

Flow Node-RED

- Créer trois fils : deux *MQTT-in* abonnés à `esp32/temperature` et `esp32/humidity` vers des widgets *gauge* et *chart*, et un *switch* Dashboard connecté à un *MQTT-out* publié sur `esp32/output` pour on/off.
- Configurer le serveur MQTT dans les nœuds avec l'IP/port du broker et déployer, puis ouvrir le Dashboard (chemin `/ui`) pour visualiser et piloter la LED.

Tests rapides

Vérifier la publication côté broker

```

1 mosquitto_sub -h IP_BROKER -t esp32/temperature -v
2 mosquitto_sub -h IP_BROKER -t esp32/humidity -v

```

Tester la commande LED

```

1 mosquitto_pub -h IP_BROKER -t esp32/output -m on
2 mosquitto_pub -h IP_BROKER -t esp32/output -m off

```